Classic problems of synchronization

Producer consumer problem

1. "**in**" used in a producer code represent the next **empty buffer**
2. "**out**" used in consumer code represent first **filled buffer**
3. "**empty**" is counting semaphore which keeps a score of no. of empty buffer
4. "**full**" is counting semaphore which scores of no. of full buffer
5. "**S**" is a binary semaphore **BUFFER**

```
void producer( void )
{
  wait ( empty );
  wait(S);
  Produce_item(item P)
  buffer[ in ] = item P;
  in = (in + 1)mod n
  signal(S);
  signal(full);

}
```

```
void consumer(void)
{
  wait ( empty );
  wait(S);
  itemC = buffer[ out ];
  out = ( out + 1 ) mod n;
  signal(S);
  signal(empty);
}
```

```
semaphore n = 0;
semaphore s = 1;
void producer()
{
      while(true)
      {
       produce();
       semWait(s);
       addToBuffer();
       semSignal(s);
       semSignal(n);
      }
}
```

```
void consumer()
{
      while(true)
      {
       semWait(s);
       semWait(n);
       removeFromBuffer();
       semSignal(s);
       consume();
      }
}
```

Which one of the following is TRUE?

(A) The producer will be able to add an item to the buffer, but the consumer can never consume it.

(B) The consumer will remove no more than one item from the buffer.

(C) Deadlock occurs if the consumer succeeds in acquiring semaphore s when the buffer is empty.

(D) The starting value for the semaphore n must be 1 and not 0 for deadlock-free operation.

Each of a set of n processes executes the following code using two semaphores a and b initialized to 1 and 0, respectively. Assume that count is a shared variable initialized to 0 and not used in CODE SECTION P.

CODE SECTION P

```
wait(a); count=count+1;
if (count==n) signal (b);
signal (a): wait (b) ; signal (b);
```

CODE SECTION Q

What does the code achieve?

A. It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P.
B. It ensures that two processes are in CODE SECTION Q at any time.
C. It ensures that all processes execute CODE SECTION P mutually exclusively.
D. It ensures that at most $n - 1$ processes are in CODE SECTION P at any time.

The shared buffer size is N. Three semaphores *empty*, *full* and *mutex* are defined with respective initial values of 0, N and 1. Semaphore *empty* denotes the number of available slots in the buffer, for the consumer to read from. Semaphore *full* denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by P, Q, R and S, in the code below can be assigned either *empty* or *full*. The valid semaphore operations are: *wait()* and *sigmal()*.

Producer:	Consumer:

do{
wait(P);
wait(mutex);
//Add item to buffer
signal(mutex);
signal(Q);
}while(1);
	do{
	wait(R);
	wait(mutex);
	//Consume item from buffer
	signal(mutex);
	signal(S);
	}while(1);

Which one of the following assignments to P, Q, R and S will yield the correct solution?

(A) P: *full*, Q: *full*, R: *empty*, S: *empty*

(B) P: *empty*, Q: *empty*, R: *full*, S: *full*

(C) P: *full*, Q: *empty*, R: *empty*, S: *full*

(D) P: *empty*, Q: *full*, R: *full*, S: *empty*

Readers Writers Problem in Operating System

The readers-writers problem is a classical problem of process synchronization, it relates to a data set such as a file that is shared between more than one process at a time. Among these various processes, some are Readers - which can only read the data set;, some are Writers - can both read and write in the data sets.

To design the code in such a manner that if one reader is reading then no writer is allowed to update at the same point of time, similarly, if one writer is writing no reader is allowed to read the file at that point of time and if one writer is updating a file other writers should not be allowed to update the file at the same point of time. However, multiple readers can access the object at the same time. We use two binary semaphores "write" and "mutex"

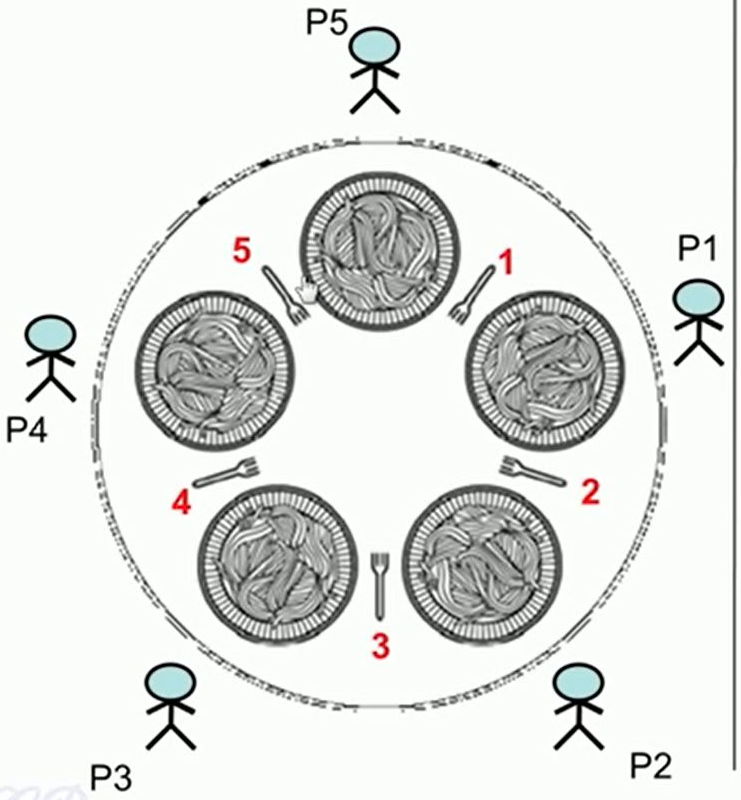
Case	Process 1	Process 2	Allowed / Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Reading	Writing	Not Allowed
Case 3	Writing	Reading	Not Allowed
Case 4	Reading	Reading	Allowed

```
static int readcount = 0;
wait (mutex);
readcount ++; // on each entry of reader increment readcoun
if (readcount == 1)
{
  wait (write);
}
signal(mutex);

--READ THE FILE?

wait(mutex);
readcount --; // on every exit of reader decrement readcount
if (readcount == 0)
{
  signal (write);
}
signal(mutex);
```

```
wait(write);
WRITE INTO THE FILE
```

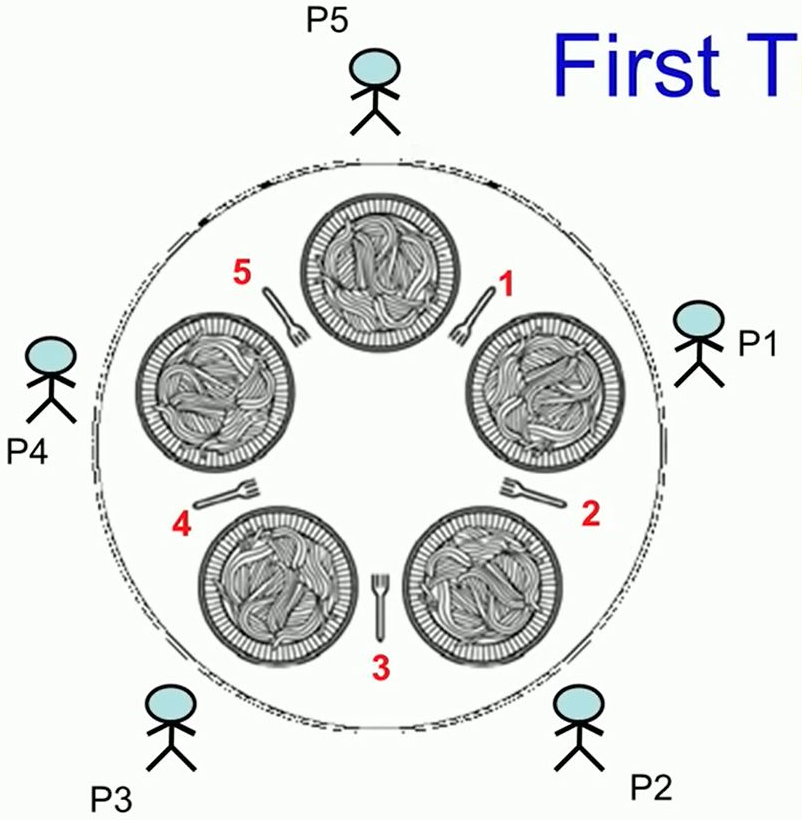
signal(write)

Dining Philosophers Problem



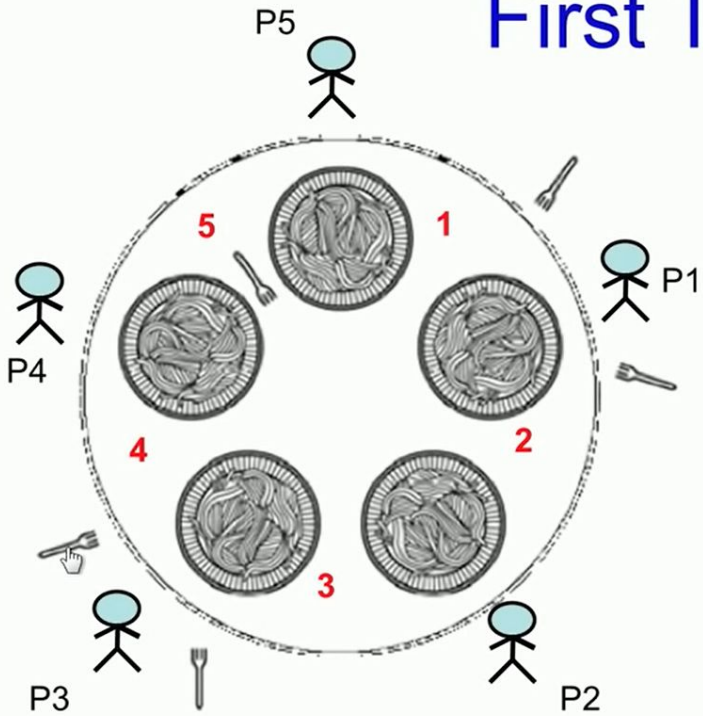
- **Philosophers either think or eat**
- To eat, a philosopher needs to hold both forks (the one on his left and the one on his right)
- If the philosopher is not eating, he is thinking.
- Problem Statement : Develop an algorithm where no philosopher starves.

First Try




```
#define N 5

void philosopher(int i){
  while(TRUE){
      think(); // for some_time
      take_fork(Ri);
      take_fork(Li);
      eat();
      put_fork(Li);
      put_fork(Ri);
  }
}
```

First Try

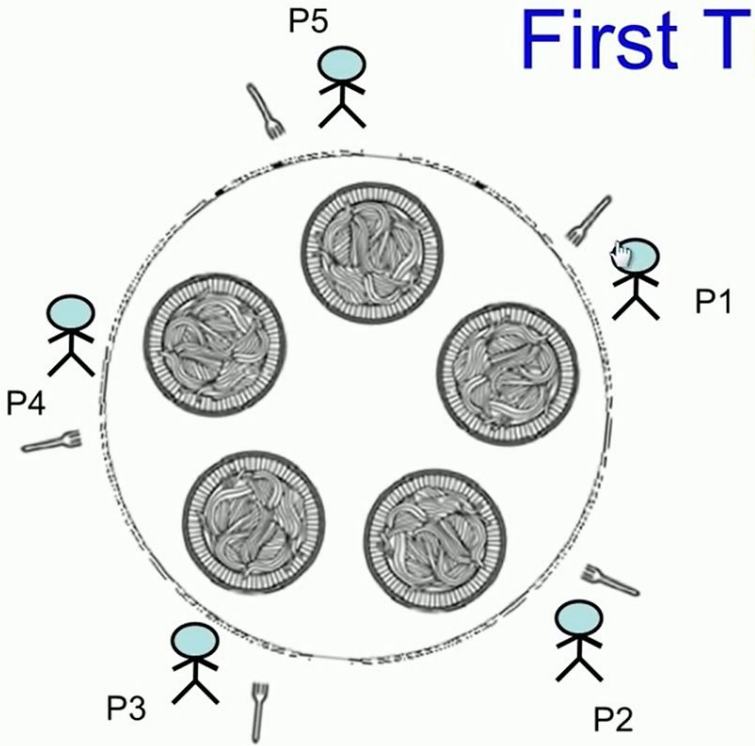



```
#define N 5

void philosopher(int i){
  while(TRUE){
     think(); // for some_time
     take_fork(Ri);
     take_fork(Li);
     eat();
     put_fork(Li);
     put_fork(Ri);
  }
}
```

What happens if only philosophers P1 and P3 are always given the priority?

P4, P5, and P2 starves… so scheme needs to be fair

First Try

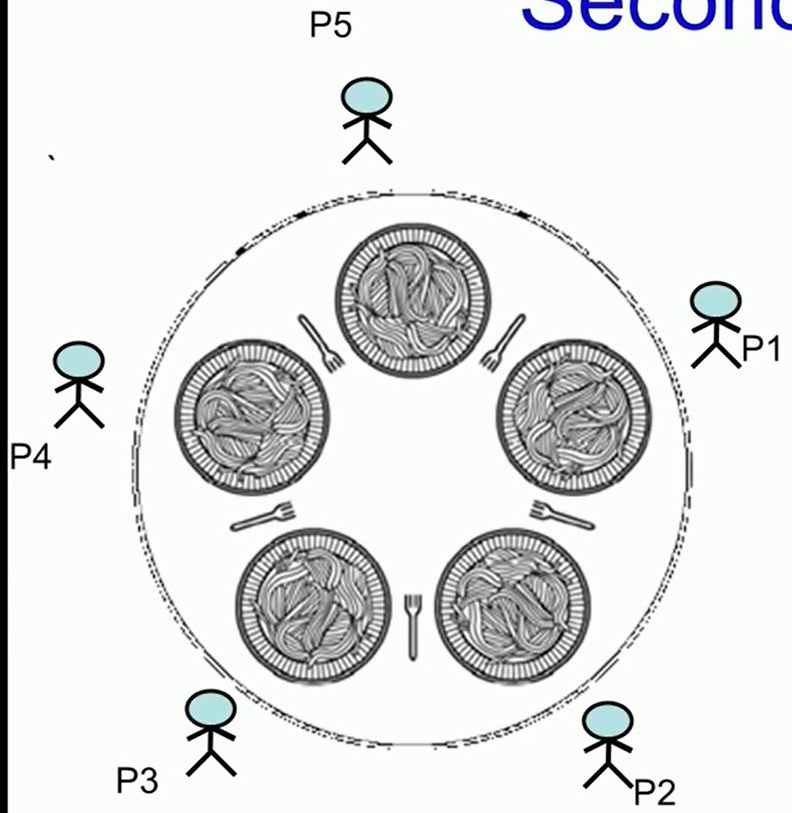



```
#define N 5

void philosopher(int i){
  while(TRUE){
     think(); // for some_time
     take_fork(Ri);
     take_fork(Li);
     eat();
     put_fork(Li);
     put_fork(Ri);
  }
}
```

What happens if all philosophers decide to pick up their right forks at the same time?

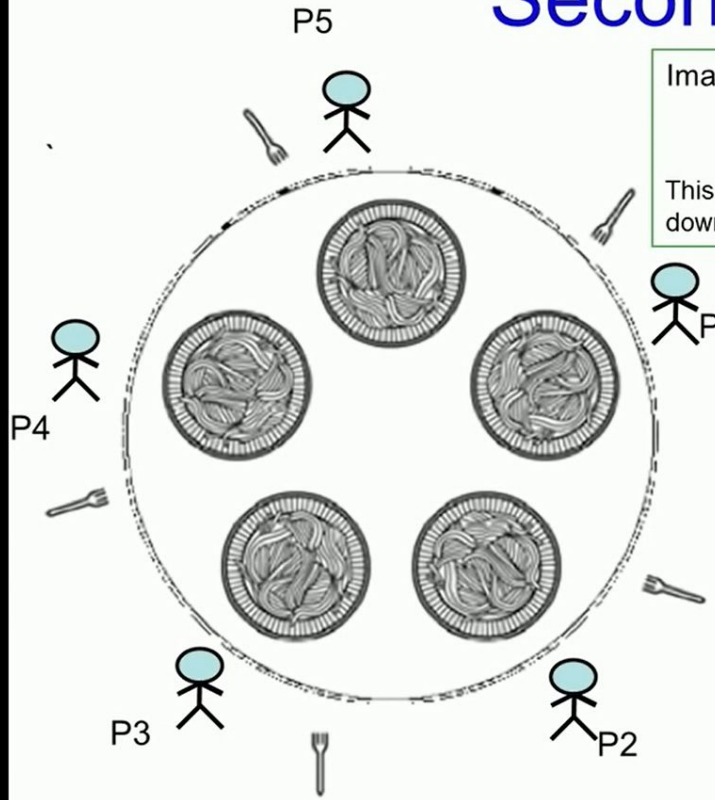
Possible starvation due to deadlock

Second try




```
#define N 5

void philosopher(int i){
  while(TRUE){
      think();
      take_fork(Ri);
      if (available(Li){
        take_fork(Li);
        eat();
        put_fork(Ri);
        put_fork(Li);
      }else{
         put_fork(Ri);
         sleep(T)
      }
}
```

Second try

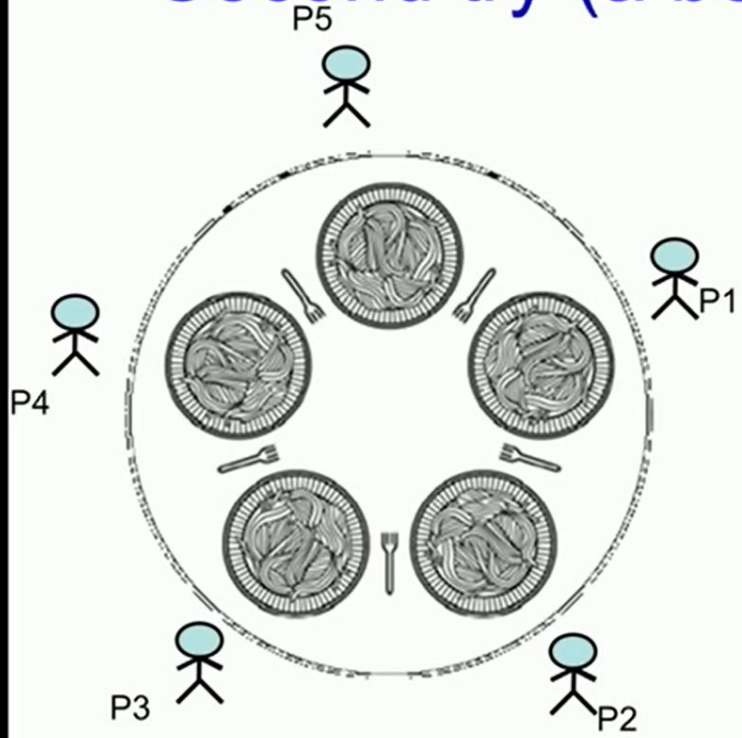


Imagine,
- All philosophers start at the same time
- Run simultaneously
- And think for the same time

This could lead to philosophers taking fork and putting it down continuously. a deadlock.

```
while(TRUE){
    think();
    take_fork(Ri);
    if (available(Li){
      take_fork(Li);
      eat();
      put_fork(Ri);
      put_fork(Li);
    }else{
       put_fork(Ri);
       sleep(T)
    }
}
```

Second try (a better solution)




```
#define N 5

void philosopher(int i){
 while(TRUE){
    think();
    take_fork(Ri);
    if (available(Li){
      take_fork(Li);
      eat();
      put_fork(Li);
      put_fork(Ri);
    }else{
       put_fork(Ri);
       sleep(random_time);
    }
}
```

Solution using Mutex

- Protect critical sections with a mutex
- Prevents deadlock
- But has performance issues
  - Only one philosopher can eat at a time

```
#define N 5

void philosopher(int i){
  while(TRUE){
     think(); // for some_time
     lock(mutex);
     take_fork(Ri);
     take_fork(Li);
     eat();
     put_fork(Li);
     put_fork(Ri);
     unlock(mutex);
  }
}
```

Solution with Semaphores

Uses N semaphores (s[1], s[2], …., s[N]) all initialized to 0, and a mutex
Philosopher has 3 states: HUNGRY, EATING, THINKING
A philosopher can only move to EATING state if neither neighbor is eating

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING){
            state[i] = EATING;
            up(s[i]);
    }
}
```

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	T	T	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	H	T	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	E	T	T
semaphore	0	0	1	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

s[i] is 1, so down will not block.
The value of s[i] decrements by 1.

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	E	T	T
semaphore	0	0	1	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

s[i] is 1, so down will not block.
The value of s[i] decrements by 1.

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	E	T	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	E	T	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING){
            state[i] = EATING;
            up(s[i]);
    }
}
```

	P1	P2	P3	P4	P5
state	T	T	E	H	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

blocked

	P1	P2	P3	P4	P5
state	T	T	E	H	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

blocked

	P1	P2	P3	P4	P5
state	T	T	T	H	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
            state[i] = EATING;
            up(s[i]);
    }
}
```

blocked

	P1	P2	P3	P4	P5
state	T	T	T	E	T
semaphore	0	0	0	0	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

blocked

	P1	P2	P3	P4	P5
state	T	T	T	E	T
semaphore	0	0	0	1	0

Solution to Dining Philosophers

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

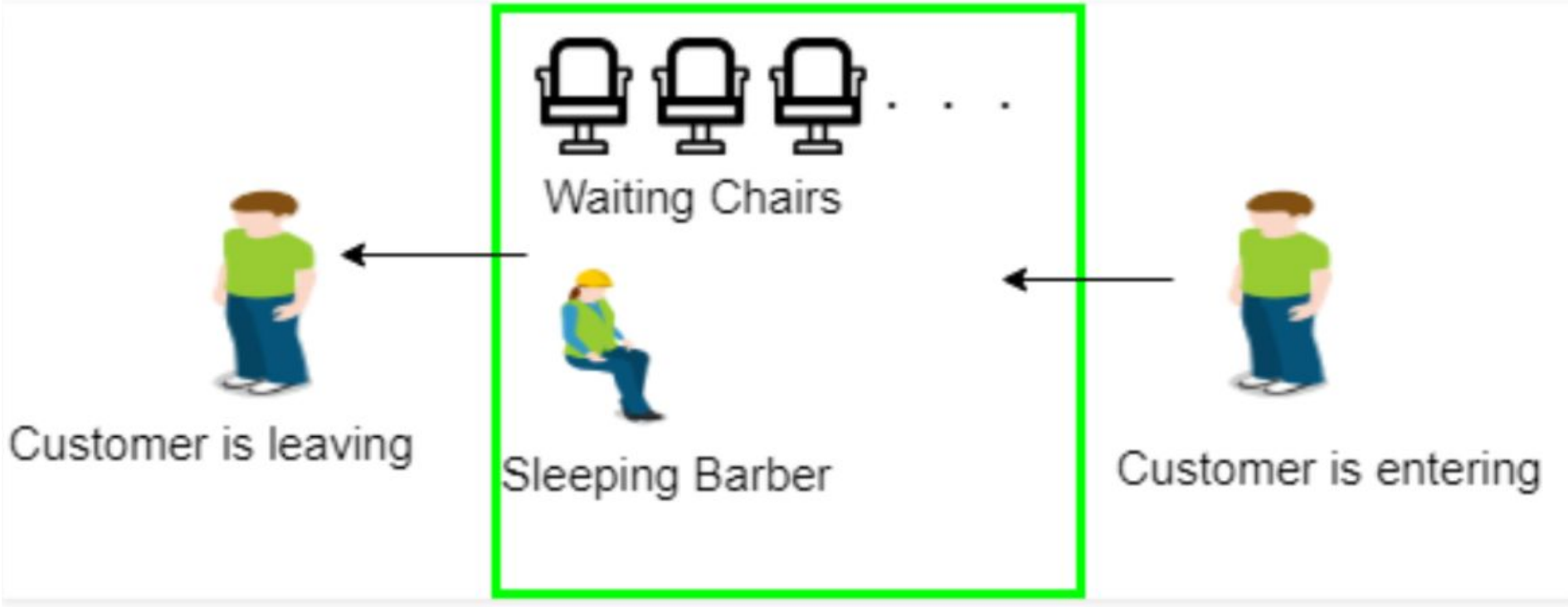
```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY &&
       state[LEFT] != EATING &&
       state[RIGHT] != EATING){
           state[i] = EATING;
           up(s[i]);
    }
}
```

wakeup

P1	P2	P3	P4	P5
T	T	T	E	T
0	0	0	0	0

Sleeping Barber problem



```
#define CHAIRS 5
semaphore customers = 0;
binary semaphore barber = 0;
binary semaphore mutex = 1;
int waiting = 0;
void barber(void) {
        while (TRUE) {
                wait(&customers);
                wait(&mutex);
                waiting = waiting - 1;
                signal(&barber);
                signal(&mutex);
                cut_hair();
        }
}
```

```
void customer(void) {
        wait(&mutex);
        if (waiting < CHAIRS) {
                waiting = waiting + 1;
                signal(&customers);
                signal(&mutex);
                wait(&barber);
                get_haircut();
        }
        else {
                signal(&mutex);
        }
}
```